

Persistencia de los Objetos en la Base de Datos



Historial de cambios

Fecha	Versión	Descripción
28/09/2025	1.0	Documento inicial
30/09/2025	1.1	- 3.1 se divide en Registro de Datos Personales y Registro de Usuario. - Se añadió la fecha de entrega (según cronograma)

Tabla de Contenido

1. Introducción	2
2. Base de Datos	2
3. Funcionalidades del Sistema	4
4. De la Implementación	6
5. Contenido de la Entrega	6
6. Información de Referencia	6

1. Introducción

En el Entregable 1 se trabajó en el modelo de clases de una Plataforma de Streaming. Generalmente una vez que el modelo de clases está acordado, suele trabajarse en el resto de la aplicación, donde se sumarán nuevas clases con otras responsabilidades.

El objetivo de esta entrega es realizar una prueba de concepto (POC), esta consiste en implementar una pequeña sección del modelo de clases que haciendo uso de alguna estrategia de conversión/mapeo nos permita persistir la información contenida en los objetos en una Base de Datos. El subconjunto de clases que usaremos en esta entrega corresponde a las relacionadas con la reseña de una Película

Se sugiere crear un nuevo proyecto para realizar este entregable.

TIP: cuando se trabaja en programación orientada a objetos, es clave la delegación de responsabilidades es decir el mecanismo por medio del cual un objeto delega en otro una determinada funcionalidad.

2. Base de Datos

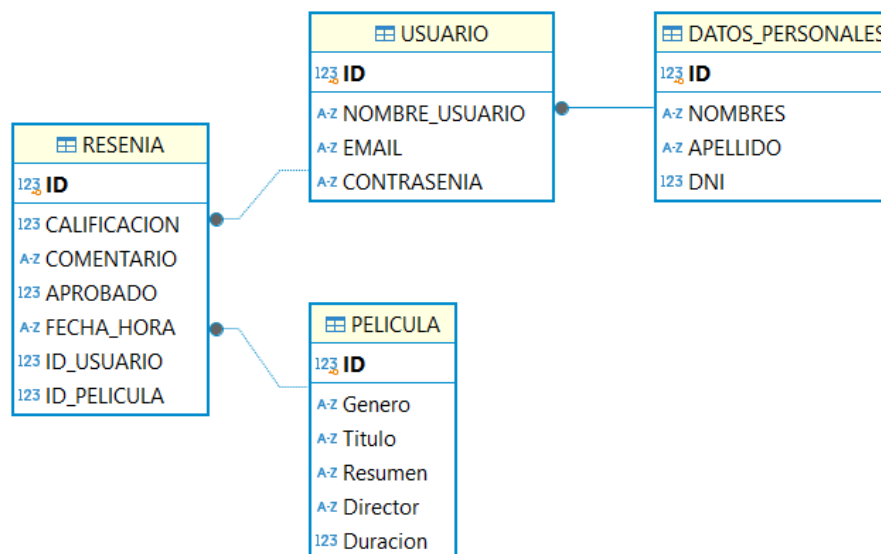
Para realizar este entregable, usaremos SQLite, una base de datos de distribución simple, que no necesita instalación, sólo necesita incorporar una librería (archivo .jar) dentro de su proyecto. Puede descargarlo desde la siguiente URL en la sección **Files**

<https://mvnrepository.com/artifact/org.xerial/sqlite-jdbc/3.50.3.0>

Si ya tiene instalado un motor de base de datos como MySQL o MariaDB también puede usarlo.

Para visualizar las tablas de la base de datos de un modo más amigable una vez creadas, puede usar cualquier herramienta visual como DBeaver, <https://sqliteviewer.app/>, etc

Para el diseño de la estructura de una base de datos se usan los diagramas de Entidades y Relaciones. Aunque lo veas similar a los diagramas UML de clases, no lo son y persiguen otro propósito. A continuación se describe el diagrama Entidad-Relación de Base de Datos que usaremos en esta entrega:



A continuación se presenta una sección de código que corresponde a la creación de las tablas (estructura interna) de la base de datos. Esta estructura es un modelo de ejemplo básico, que puede ajustarse de acuerdo a su diagrama UML de clases si fuera necesario:

```
/**
 * Este metodo se encarga de la creación de las tablas donde se
 * almacenara la
 * informacion de los objetos. Una vez establecida una conexion, debería
 * ser lo proximo a ser ejecutado.
 *
 * @param connection objeto conexion a la base de datos SQLite
 * @throws SQLException
 */

private void creacionDeTablasEnBD(Connection connection) throws SQLException
{
    Statement stmt;
    stmt = connection.createStatement();
    String sql=" CREATE TABLE IF NOT EXISTS DATOS_PERSONALES ("+
        "ID INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT,"+
        "NOMBRES TEXT(100) NOT NULL,"+
        "APELLIDO TEXT(100) NOT NULL,"+
        "DNI INTEGER NOT NULL"+
        ");";
    stmt.executeUpdate(sql);

    sql=" CREATE TABLE IF NOT EXISTS PELICULA ("+
        "ID INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT,"+
        "GENERO TEXT(1) NOT NULL,"+
        "TITULO TEXT(100) NOT NULL,"+
        "RESUMEN TEXT(500),"+
        "DIRECTOR TEXT(100) NOT NULL,"+
        "DURACION REAL NOT NULL"+
        ");";

    stmt.executeUpdate(sql);

    sql=" CREATE TABLE IF NOT EXISTS USUARIO ("+
        "ID INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT,"+
        "NOMBRE_USUARIO TEXT NOT NULL,"+
        "EMAIL TEXT NOT NULL,"+
        "CONTRASENIA TEXT NOT NULL, ID_DATOS_PERSONALES INTEGER NOT
NULL,"+
        "CONSTRAINT USUARIO_DATOS_PERSONALES_FK FOREIGN KEY (ID)
REFERENCES DATOS_PERSONALES(ID)"
        ");";

    stmt.executeUpdate(sql);
}
```

```
sql=" CREATE TABLE IF NOT EXISTS RESENIA (" +
      "ID INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT," +
      "CALIFICACION INTEGER NOT NULL," +
      "COMENTARIO TEXT(500)," +
      "APROBADO INTEGER DEFAULT (1) NOT NULL," +
      "FECHA_HORA DATETIME NOT NULL," +
      "ID_USUARIO INTEGER NOT NULL," +
      "ID_PELICULA INTEGER NOT NULL," +
      "CONSTRAINT RESENIA_USUARIO_FK FOREIGN KEY (ID_USUARIO)
REFERENCES USUARIO(ID)," +
      "CONSTRAINT RESENIA_PELICULA_FK FOREIGN KEY (ID_PELICULA)
REFERENCES PELICULA(ID)" +
      ");";

stmt.executeUpdate(sql);

stmt.close();

}
```

3. Funcionalidades del Sistema

Se debe entregar una aplicación Java que se ejecute por línea de comando, **sin** interfaz gráfica de ventanas y que permita realizar las siguientes operaciones:

1. Registrar Datos Personales.

- Se solicita al usuario que ingrese uno a uno sus datos personales. Una vez ingresados todos los datos, se le muestra al usuario todos los datos que acaba de ingresar. Además el sistema pregunta si los datos son correctos, y si el usuario confirma que son correctos se guardan en la Base de Datos.
- Validaciones:
 - No se permite el ingreso de números en los campos nombres y apellido.
 - Se debe verificar que el DNI sea único
 - Se deben ingresar todos los datos.
 - El sistema debe informar los datos erróneos

Tenga en cuenta que si ud. modeló otros campos para datos personales, o su estructura difiere considerablemente de la estructura propuesta, puede hacer adecuaciones. Consulte con su docente asignado cual es la mejor forma de reestructurar esta tabla de la base de datos.

2. Registrar Usuario.

- Se muestra el listado de personas guardadas en el sistema para elegir una persona de ese listado, la cual quedará asociada al Usuario que se está creando.
- Se solicita al usuario que ingrese uno a uno sus datos necesarios para la creación del usuario en "la plataforma". Una vez ingresados todos los datos, se le muestra al usuario todos los datos que acaba de ingresar. Además el sistema pregunta si los

datos son correctos, y si el usuario confirma que son correctos se guardan en la Base de Datos.

- Validaciones:
 - Se debe verificar que el email ingresado tenga el formato xxx@yyy.
 - Se deben ingresar todos los datos.
 - El sistema debe informar los datos erróneos

3. Registrar Película

- Se solicita al usuario que ingrese uno a uno sus datos asociados a la película. Una vez ingresados los datos, el sistema muestra la información en la consola y luego de la confirmación de que los datos son correctos, se procede a guardarlos en la Base de Datos.
- Validaciones:
 - Los valores de Género deben ser tomados/validados a partir de un Enumerativo que se carga en memoria al inicio de la aplicación.
 - Todos los datos son requeridos a excepción del Resumen.
 - El sistema debe informar por consola los datos erróneos.

4. Listar Usuarios

- Muestra en pantalla información de los usuarios existentes en la Base de Datos, incluyendo los datos personales.
- La opción de listar debe permitir mostrar los datos ordenados por Nombre de Usuario o por Email. Se espera que use **alguno de los mecanismos de ordenación ofrecidos con interfaces** Java desarrollados en la teoría.

5. Listar Películas

- Muestra en la consola toda la información de las películas existentes en la Base de Datos.
- La opción de listar debe permitir mostrar los datos ordenados por Título o por Género o por Duración. Se espera que use **alguno de los mecanismos de ordenación ofrecidos con interfaces** Java desarrollados en la teoría.

6. Registrar Reseña

- Se compone de los siguientes pasos:
 - Ingresar un usuario y una contraseña que el sistema valida.
 - Si la validación es exitosa, mostrar un listado numerado con los títulos de las películas disponibles.
 - El usuario indica un número de película válido.
 - A continuación se solicitan todos los datos de la reseña (calificación, comentario).
 - Si el usuario confirma, se guarda en la base de datos.

7. Aprobar Reseña

- Muestra en pantalla las reseñas NO aprobadas.
- Se solicita al usuario el número de reseña a aprobar.
- Se valida que la reseña seleccionada exista, caso contrario se informa al usuario.

- Se muestra la reseña seleccionada.
- Si el usuario confirma, se procede a la aprobación

4. De la Implementación

- A. Se espera que se implemente una aplicación de escritorio en Java, donde la interacción se realiza a través de la línea de comandos.
- B. Se espera que el código esté modularizado.
- C. No se requiere que implemente todo su modelo de clases, **sólo un conjunto mínimo de clases** que permita realizar las operaciones requeridas en la sección anterior.
- D. Debe usar los mecanismos de ordenación desarrollados en teoría para la implementación de los listados ordenados.
- E. Debe implementar el patrón DAO desarrollado en la teoría.

5. Contenido de la Entrega

La entrega deberá realizarse el **27/10**, incluye un archivo ZIP que contiene lo siguiente:

- A. El proyecto Java con el código fuente
- B. Un archivo .jar ejecutable con la aplicación
- C. Un archivo readme con cualquier particularidad o información que considere necesario tener en cuenta para ejecutar su aplicación.

6. Información de Referencia

Uso de SQLite

https://www.tutorialspoint.com/sqlite/sqlite_java.htm

Statements:

<https://docs.oracle.com/javase/tutorial/jdbc/basics/processingsqlstatements.html>

Prepared Statements:

<https://docs.oracle.com/javase/tutorial/jdbc/basics/prepared.html>

Recuperando datos de Result Set:

<https://docs.oracle.com/javase/tutorial/jdbc/basics/retrieving.html>